Spring MVC Request Flow

Till now we saw how Spring Boot starts: SpringApplication.run()

That process:

- prepares the environment
- creates ApplicationContext
- loads auto configuration
- starts the application

Once Boot finishes startup, the application is ready to handle requests.

So the next question is:

- What happens when a client sends an HTTP request to our application?

To answer this we must understand Spring MVC architecture.

HIGH LEVEL REQUEST FLOW

What Spring MVC Actually Is

It is the web framework inside Spring that implements the Model-View-Controller pattern.

Its responsibility is to handle HTTP requests and responses.

Spring infrastructure includes components like:

- DispatcherServlet (front controller)
- HandlerMapping
- HandlerAdapter
- Controller
- ViewResolver
- HttpMessageConverters

So Spring MVC is basically the web request processing engine.

It is based on the Front Controller design pattern.
Front controller means:

- All HTTP requests first go to a central component before reaching controllers.

In Spring MVC, that central component is called:

- DispatcherServlet

A typical Spring Boot web application follows three layered architecture:

- Controller Layer
- Service Layer
- Repository Layer

Each layer has a specific responsibility.

- Controller → handles HTTP requests
- Service → contains business logic
- Repository → interacts with database

However, before reaching the controller, requests must pass through the Spring MVC infrastructure.

WEB SERVER RECEIVES REQUEST

Spring Boot uses an embedded server.

By default: Apache Tomcat

When Boot starts, it automatically starts an embedded Tomcat server.
Tomcat listens on a port (default 8080).

When a client sends a request:

- GET /users

Tomcat receives it.

But Tomcat does not know about controllers.

Instead, it forwards the request to a special servlet called:

- DispatcherServlet

DISPATCHERSERVLET (THE HEART OF SPRING MVC)

DispatcherServlet is the front controller in Spring MVC.

Front controller means:

- All requests first go to a single central component.

DispatcherServlet coordinates the entire request lifecycle.

It does not handle business logic.

Instead, it delegates tasks to other components.

HANDLER MAPPING

When DispatcherServlet receives a request, the first question is:
- which controller method should handle this request?

To answer this Spring MVC uses HandlerMapping.

HandlerMapping maps a request URL to a controller method.

Example:

- GET /users

Mapped to:

- UserController.getUsers()

Spring MVC supports several handler mappings, but the most commonly used one is:
- RequestMappingHandlerMapping

This mapping reads annotations like:

- @GetMapping
- @PostMapping
- @RequestMapping

HANDLER ADAPTER

Once Spring identifies the correct controller method, it must execute it.

However, controllers can have different method signatures.

Example:

- @GetMapping("/users")
  public List<User> getUsers()

Another controller might look like:

- @GetMapping("/users/{id}")
  public User getUser(@PathVariable Long id)

Spring needs a way to invoke these methods generically.

This is done by HandlerAdapter.

HandlerAdapter knows how to call controller methods.

The most commonly used adapter is:
- RequestMappingHandlerAdapter

CONTROLLER EXECUTION

Now the controller method executes.

Example:

```
@GetMapping("/users")
public List<User> getUsers() {
    return userService.getUsers();
}
```

Controller should not contain business logic.

Instead, it delegates to the Service layer.

SERVICE LAYER

Service layer contains business logic.

Example:

- UserService

Responsibilities include:

- validations
- business rules
- orchestration of operations

Example:

```
public List<User> getUsers() {
    return userRepository.findAll();
}
```

REPOSITORY LAYER

Repository layer interacts with the database.

Spring Boot commonly uses:
- Spring Data JPA

Repositories perform:

- CRUD operations
- database queries

Example:

UserRepository extends JpaRepository

RETURNING THE RESPONSE

After controller returns data, Spring MVC processes the response.

Example return:

List<User>

- Java Object
- HttpMessageConverter
- JSON

Spring must convert this Java object into JSON.

This is handled by HttpMessageConverters.

The default converter for JSON is provided by:
- Jackson

Jackson converts Java objects into JSON automatically.

RESPONSE FLOW

- Controller
- HandlerAdapter
- DispatcherServlet
- Tomcat
- Client

COMPLETE REQUEST LIFECYCLE

- Client
- Tomcat
- DispatcherServlet
- HandlerMapping
- HandlerAdapter
- Controller
- Service
- Repository
- Database
- Controller Response
- HttpMessageConverter
- JSON Response
- Client

WHERE AUTO CONFIGURATION FITS

We saw:

Auto configuration loads:
- WebMvcAutoConfiguration

This automatically configures:

- DispatcherServlet
- HandlerMapping
- HandlerAdapter
- MessageConverters

So developers only write:

- @Controller
- @GetMapping

Spring Boot configures the infrastructure.

REST APIs in Spring Boot

WHAT IS AN API

API stands for Application Programming Interface.

It is a contract that allows two software systems to communicate.

Example:

Client sends request: GET /weather

Server returns: temperature = 30
city = Delhi

WHAT IS REST API

REST stands for:
Representational State Transfer.

But practically REST APIs mean:

Using HTTP + URLs + JSON to communicate between client and server

-> Resource based

-> Stateless

-> Uses HTTP methods

-> Returns representation (usually JSON)

RESOURCES IN REST

Example resources:

users
orders
products

```
GET /users
GET /users/1
POST /users
DELETE /users/1
```

users is a collection resource.

users/1 is a specific resource.

HTTP REQUEST STRUCTURE

Method
URL
Headers
Body

Example:

POST /users

Headers:

Content-Type: application/json

Body:

```
{
"name": "Rahul"
}
```

Method tells server what operation to perform.

URL identifies resource.

Headers contain metadata.

Body contains data.

HTTP RESPONSE STRUCTURE

Status Code
Headers
Body

Example:

200 OK

Body:

```
{
"id": 1,
"name": "Rahul"
}
```

HTTP METHODS

GET → retrieve data
Example:
GET /users

POST → create new resource
POST /users

PUT → replace entire resource
PUT /users/1

PATCH → update partial resource
PATCH /users/1

DELETE → remove resource
DELETE /users/1

PUT replaces entire object.

PATCH updates only specific fields.

URL PARAMETERS

Path Variables

Used to identify specific resource.

Example:
GET /users/5

Here 5 is path variable.

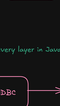
Query Parameters

Used for filtering or searching.

Example:
GET /users?age=25

Example with pagination:
GET /users?page=1&size=10

Spring Data JPA



"Every layer in Java persistence exists because the previous one had problems."

1. WHY JDBC EXISTED?

Problem Before JDBC

Early days:

Every database had its own API
Oracle API ≠ MySQL API ≠ PostgreSQL API

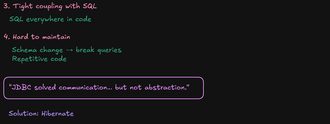
Problem:

--> No standard way to talk to databases

Solution:

-> JDBC (Java Database Connectivity)

-> What JDBC solved:
Standard API to connect DB
Send SQL queries
Get results



Example:

```
Connection conn = DriverManager.getConnection(...);
PreparedStatement ps = conn.prepareStatement("SELECT * FROM users");
ResultSet rs = ps.executeQuery();
```

What JDBC DID NOT solve:
This is the MOST IMPORTANT part

1. No Object Mapping

```
User user = new User();
user.setName(rs.getString("name"));
```

-> Manual mapping every time

2. Boilerplate

Connection handling
Statement handling
Exception handling

3. Tight coupling with SQL

SQL everywhere in code

4. Hard to maintain

Schema change → break queries
Repetitive code

"JDBC solved communication... but not abstraction."

Solution: Hibernate

2. WHY HIBERNATE (ORM)?

What Hibernate solved

1. Object ↔ Table Mapping

```
userRepository.save(user);
```

-> No SQL needed

2. Automatic SQL generation

Hibernate generates:

```
INSERT INTO users ...
```

3. Relationships

@OneToMany

-> Handles joins

Problems with Hibernate

1. No Standard

Hibernate = one implementation

2. Vendor Lock-in

```
Session session = sessionFactory.openSession();
```

-> Hibernate-specific API

3. Multiple ORM tools

Hibernate
EclipseLink
OpenJPA

-> All different

4. No portability

Switching ORM = rewrite code

"Hibernate solved mapping... but created fragmentation."

3. WHY JPA?

Problem after Hibernate

-> Too many ORMs, no standard

Solution: Java Persistence API

What JPA solved

1. Standardization

All ORMs follow same rules

2. Common API

```
entityManager.persist(user);
```

3. Portability

Switch ORM without changing code

4. Standard annotations

```
@Entity
@Id
@OneToMany
```

5. Standard query language

JPQL

Problems with JPA

1. Too much boilerplate

```
EntityManager em = ...
em.persist(user);
```

2. DAO layer needed

You still write:

```
UserDao.save(user);
```

3. Complex setup

Config
Transactions
EntityManager handling

4. Repetitive code

CRUD methods again and again

"JPA standardized ORM... but still required a lot of code."

Solution: Spring Data JPA

4. WHY SPRING DATA JPA?

1. No DAO implementation

```
public interface UserRepository extends JpaRepository<User, Long> {}
```

2. Auto CRUD

```
save()
findAll()
delete()
```

3. Query generation

```
findByName(String name);
```

4. Pagination & sorting

5. Integration with Spring

Transactions
Dependency injection

COMPLETE EVOLUTION SUMMARY

Step 1: JDBC

-> Solved: DB connection
-> Problem: No abstraction

"Each layer didn't replace the previous one... It fixed the problems of the previous one."

Step 2: Hibernate

-> Solved: Object mapping
-> Problem: No standard

Step 3: JPA

-> Solved: Standardization
-> Problem: Boilerplate

Step 4: Spring Data JPA

-> Solved: Boilerplate

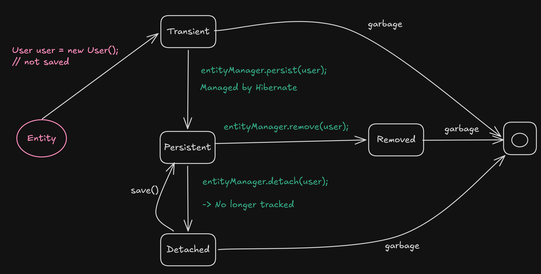
RUN POSTGRES USING DOCKER

Instead of installing PostgreSQL locally, we'll use Docker -> clean, isolated, production-like.

```
docker run --name postgres-container \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=pass \
  -e POSTGRES_DB=testdb \
  -p 5432:5432 \
  -d postgres
```

ENTITYMANAGER & ENTITY LIFECYCLE

```
User user = new User();
// not saved
```



PERSISTENCE CONTEXT

Persistence context ensures one object per DB row.

User(id=1, name=Aniket)

```
User user = repo.findById(1).get();
user.setName("Updated");
```

No save() -> because Hibernate tracks entity

Persistence context ensures one object per DB row.

DIRTY CHECKING + @Transactional

Hibernate automatically detects changes — this is dirty checking.

```
@Transactional
public void updateUser() {
    User user = repo.findById(1).get();
    user.setName("New Name");
}
```

-> Transaction starts
-> Entity loaded into memory
-> Change detected
-> Hibernate runs UPDATE

No save() needed — Hibernate syncs automatically.

QUERY METHODS in JPA

Spring Data JPA can generate queries just from method names.

```
List<User> findByName(String name);

List<User> findByNameAndEmail(String name, String email);

List<User> findByNameContaining(String keyword);
```

What happens:

Spring parses method → generates SQL

LIMITATION

Complex queries → not possible
Long method names

@Query (CUSTOM JPQL)

When derived queries are not enough, we write JPQL

```
@Query("SELECT u FROM User u WHERE u.name = :name")
List<User> findUsersByName(@Param("name") String name);
```

JPQL	SQL
Uses entity Uses fields	Uses table Uses columns

Example:
SELECT u FROM User u
NOT:

SELECT * FROM users

IMPORTANT LINE
"JPQL works on objects, not tables"

NATIVE QUERY

Sometimes JPQL is not enough — then we use native SQL.

```
@Query(value = "SELECT * FROM users WHERE name = ?1", nativeQuery = true)
List<User> findByNameNative(String name);
```

When to use:

- Complex joins
- DB-specific features
- Performance tuning

DOWNSIDE

- DB dependent
- Not portable

MODIFYING QUERY

For update/delete queries, we use @Modifying.

```
@Modifying
@Transactional
@Query("UPDATE User u SET u.name = :name WHERE u.id = :id")
int updateUserName(@Param("id") Long id, @Param("name") String name);
```

Why needed?

-> JPQL is SELECT by default

IMPORTANT
Must use @Transactional
Must use @Modifying

IMPORTANT LINE
"Without @Modifying, update queries will fail."

What is an Exception?

Runtime problem or An unexpected event that interrupts normal flow of program

Examples:

User not found
Invalid input

Types of Exceptions (Java)

1. Checked Exceptions (Compile-time)
   Must handle or declare

   Example:
   IOException
   SQLException

2. Unchecked Exceptions (Runtime)
   Occur at runtime
   Most common in Spring Boot

   Example:
   NullPointerException
   IllegalArgumentException

In backend systems, most real problems are runtime exceptions.

What Happens Without Handling?

In a Spring Boot API:

Server throws exception
User gets ugly response like:

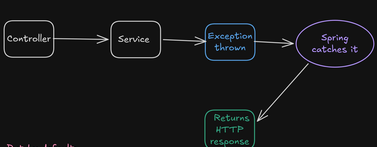
```
{
  "timestamp": "...",
  "status": 500,
  "error": "Internal Server Error",
  "trace": "Full stack trace "
}
```

Problems:

Leaks internal details
Bad frontend experience
No consistency

Exception in Spring Boot

In Spring Boot:
-> Exceptions flow like this:



But by default:

Not clean
Not controlled
Not standardized

Goal of Exception Handling

1. Meaningful error response
2. Centralized handling
3. Clean controller code
4. Proper HTTP status codes

Types of Exception Handling in Spring Boot

1. Local (try-catch in method)

```
@GetMapping("/{id}")
public User getUser(@PathVariable Long id) {
    try {
        return userService.getUser(id);
    } catch (Exception e) {
        throw new RuntimeException("Something went wrong");
    }
}
```

Problems:

Boilerplate
Repeated everywhere

"Not scalable"

2. Controller Level (@ExceptionHandler)

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<String> handleUserNotFound(UserNotFoundException ex) {
    return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
}
```

Better but:

Only for one controller

3. Global Exception Handling (Recommended)

Step 1: Create Custom Exception
Step 2: Throw Exception in Service
Step 3: Create Global Handler

What @Valid does

@Valid tells Spring to run Bean Validation (Jakarta Validation) on the request body before your controller method runs.

If something fails, Spring does not call your method body with bad data; it throws MethodArgumentNotValidException (typically → 400 Bad Request).

1. Constrain the DTO fields

Example rules you might add on CreateUserDto (name, email):

@NotBlank — required, non-empty after trim
@Email — basic email shape
@Size(max = ...) — length limits
Conceptually:

```
import jakarta.validation.constraints.*;
public class CreateUserDto {
    @NotBlank
    @Size(max = 100)
    private String name;
    @NotBlank
    @Email
    @Size(max = 255)
    private String email;
}
```

For CreateOrderDto, e.g. @NotBlank on productName, maybe @Size(max = 200)

2. Trigger validation in the controller

Put @Valid on the @RequestBody parameter wherever you accept that DTO:

```
@PostMapping
public ResponseEntity<UserDto> createUser(@Valid @RequestBody CreateUserDto createUserDto) {
    // only runs if validation passed
}
```

Same idea for @PutMapping / @PatchMapping / CreateOrderDto on order creation.

3. What the client gets by default

Without a custom handler, Spring Boot usually returns a 400 with a JSON body describing binding/validation errors (field names and default messages).
That is often enough for APIs.

4. Optional: custom error shape (@RestControllerAdvice)

If you want one consistent JSON shape (e.g. code, message, fieldErrors), add:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<...> handleValidation(MethodArgumentNotValidException ex) {
    // read ex.getBindingResult().getFieldErrors()
}
```

Spring Security

Why Spring Security Came

Before Spring Security:

Problems in Java web apps

Authentication handled manually (login servlet, session logic)

Authorization scattered across code (if(role == ADMIN))

No standard way to:
- Prevent CSRF
- Handle password hashing securely
- Manage sessions properly

Security logic tightly coupled with business logic

Result:

- Code duplication
- Security bugs (very common)
- Hard to maintain & audit

Spring Security solved:

- Centralized security framework
- Declarative security (annotations/config)
- Pluggable authentication mechanisms
- Built-in protections (CSRF, session fixation, etc.)

Core Philosophy

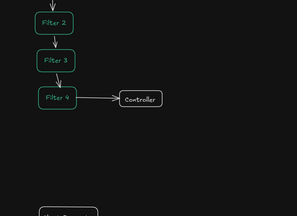
Spring Security is based on:

"Everything is a filter" + "Security as cross-cutting concern"

It integrates with Spring via:

Filters (Servlet layer)
AOP (method-level security)

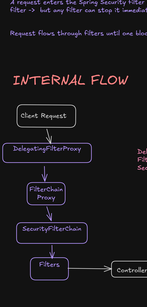
Where Spring Security Sits



Key Idea:
Spring Security intercepts the request before your controller ever sees it.

What is Spring Security Filter Chain?

Spring Security = a chain of filters that every request passes through before reaching your controller



Order matters a LOT.

Each filter can:

Stop the request
Modify the request
Pass it forward

Any filter can block the request : that's how security is enforced.

A request enters the Spring Security filter chain and moves forward filter by filter -> but any filter can stop it immediately if security checks fail.

Request flows through filters until one blocks it or it reaches the controller

INTERNAL FLOW

DelegatingFilterProxy -> entry point
FilterChainProxy -> selects chain
SecurityFilterChain -> actual filters

Authentication = Who are you?

Authentication process

Security Filter Chain

Spring Security never directly talks to DB — it uses UserDetailsService.

A request enters the Security Filter Chain.
One of the filters triggers authentication.

Spring calls AuthenticationManager, which delegates to AuthenticationProvider.

AuthenticationProvider calls UserDetailsService to fetch user from database and uses PasswordEncoder to verify password.

If successful, the user is stored in SecurityContext.

Then authorization happens, and finally the request reaches the controller

Security context

SecurityContext is where Spring Security stores the currently authenticated user.

Who is the current user for this request?

SecurityContextHolder.getContext().getAuthentication()

After successful login, Spring stores the user inside SecurityContext, and every filter or controller can access it from there.

JWT Authentication in Spring Boot

AUTH APIs (Public — no token required)

```
POST /api/v1/auth/register    → Register new user
POST /api/v1/auth/login       → Login → returns JWT
```

USER APIs (Logged-in USER only)

Identity comes from JWT (no {id})

```
GET /api/v1/users/me          → Get current user profile
PUT /api/v1/users/me          → Update own profile
```

ADMIN USER MANAGEMENT APIs

```
GET    /api/v1/users          → Get all users
GET    /api/v1/users/{id}     → Get user by id
DELETE /api/v1/users/{id}     → Delete user
```

ORDER APIs

USER (own data only)

```
POST   /api/v1/orders         → Create order
GET    /api/v1/orders/my      → Get my orders
GET    /api/v1/orders/my/{id} → Get my specific order
```

ADMIN (full access)

```
GET    /api/v1/orders         → Get all orders
GET    /api/v1/orders/user/{id}   → Get orders of specific user
DELETE /api/v1/orders/{id}    → Delete any order
```

Testing in Spring Boot

Unit Testing in Spring Boot

Unit testing means testing a single unit of code in isolation.

In Spring Boot, that usually means testing:

a service class
utility class
mapper
validation logic
business logic

without starting the full Spring application.

Modern Testing Stack

For pure unit tests:

JUnit 5
Mockito
AssertJ

are enough.

What Should NOT Be Unit Tested?

Usually avoid unit testing:

JPA repositories
Spring annotations
Lombok generated code
simple getters/setters

Those are better covered by integration tests.

Best Practices

1. Test Behavior, Not Implementation

   Good:

   verify(paymentService)

   Bad:

   verify(privateMethod)

2. One Scenario Per Test

   Bad:

   testEverything()

   Good:

   shouldCreateOrder()
   shouldThrowExceptionWhenStockUnavailable()

3. Use Clear Naming

   Recommended naming:

   shouldXWhenY

Examples:

   shouldReturnUserWhenUserExists()

   shouldThrowExceptionWhenUserMissing()

AAA Pattern

Arrange

Prepare data.

when(...)

Act

Call method.

service.method()

Assert

Verify output.

assertThat(...)

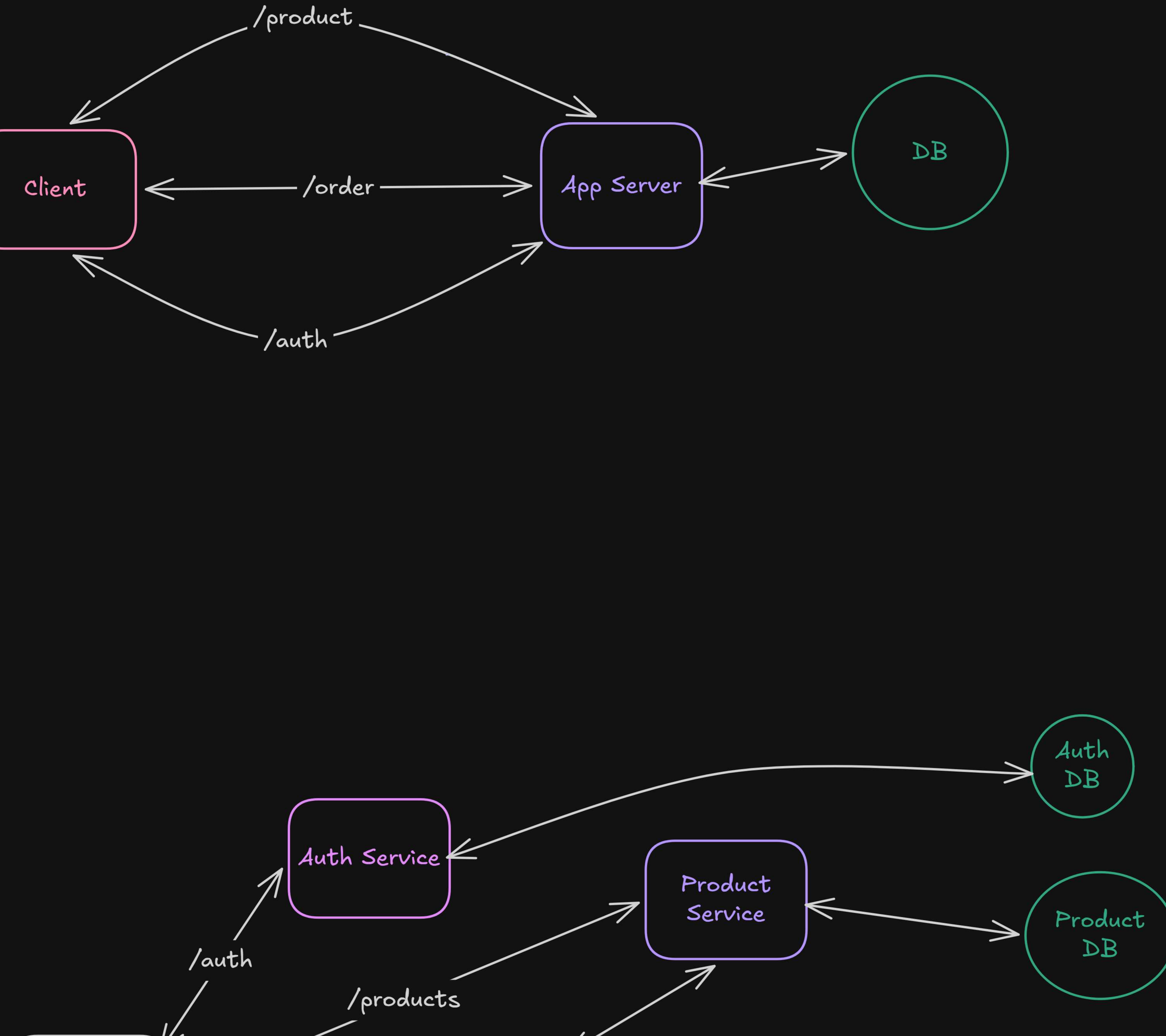
Integration Testing in Spring Boot

Integration testing verifies that multiple parts of your application work together correctly.

Instead of testing one class in isolation (unit testing), integration tests check things like:

Controller + Service + Repository together
Database interaction
REST APIs
Security configuration
External APIs
Full application flow

Unit Test vs Integration Test

Unit Test	Integration Test
Tests single class	Tests multiple layers together
Uses mocks heavily	Uses real components
Very fast	Slower
No real DB	Usually real/test DB
Mockito focused	Spring context focused
Example: service method	Example: API + DB flow

Building Event-Driven Microservices with Spring Boot & Kafka